

PRODUCT REQUIREMENTS DOCUMENT

P_205

AI-Based Crop Disease Detection System

Professional PRD for a 38-Class Crop Disease Detection Web Application

Project Type	AI/ML + Full-Stack Web Application
Frontend	React.js + Vite
Backend	Node.js + Express.js
ML Service	Python
Model	Deep Learning / CNN / Transfer Learning
Dataset	Crop Disease Dataset — 38 Classes
Database	Not required for MVP
Document Version	1.0
Status	Development

1. Executive Summary

The AI-Based Crop Disease Detection System is a web application designed to identify diseases affecting crops from images of plant leaves. A user uploads an image through the web interface, the image is processed by a machine-learning inference service, and a trained image-classification model predicts the disease class and confidence score.

The system is designed as a preliminary decision-support and educational tool for farmers, agriculture students, researchers, gardeners, and other agricultural users. The first release will support 38 disease/healthy classes from the selected crop disease dataset.

2. Problem Statement

Crop diseases can reduce crop productivity and quality. Manual identification may require agricultural expertise and can be difficult when symptoms look similar across diseases. The proposed system uses computer vision and deep learning to automate preliminary disease classification from plant images.

The application will provide disease identification, confidence information, disease descriptions, and general preventive guidance. It is not intended to replace professional agricultural diagnosis.

3. Product Vision

Vision: To provide an accessible AI-powered platform that helps users identify possible crop diseases from plant images quickly and understand the detected condition through clear, easy-to-understand information.

4. Product Objectives

- Allow users to upload a plant or leaf image.
- Validate the uploaded image before processing.
- Send the image to the ML inference service.
- Preprocess the image and classify it using the trained model.
- Return the predicted disease and confidence score.
- Display crop and disease information.
- Provide basic preventive recommendations.
- Handle invalid images, network errors, and service failures gracefully.
- Provide a responsive interface for desktop and mobile users.

5. Target Users

- **Farmers:** Users seeking preliminary information about possible crop diseases.
- **Agriculture Students:** Students learning about plant diseases and AI applications.
- **Researchers:** Users studying computer vision and agricultural machine learning.
- **Gardeners:** Users interested in identifying diseases affecting plants in home gardens.

6. Product Scope

6.1 In Scope

- React/Vite web interface.

- Plant/leaf image upload and preview.
- Image validation.
- 38-class disease classification.
- ML inference through a Python service.
- Node.js/Express application backend.
- Prediction confidence score.
- Disease information and general recommendations.
- Responsive UI and error handling.

6.2 Out of Scope for MVP

- User authentication and profiles.
- Online payments.
- MongoDB or other persistent database.
- Expert consultation marketplace.
- Native mobile application.
- Weather integration.
- Real-time disease outbreak monitoring.

7. Supported Disease Classification

The ML system will support exactly 38 classes from the project's prepared dataset. The final implementation should keep the class names in a centralized configuration such as **class_names.json** so that model output and UI labels remain synchronized.

The exact 38 class names should be taken from the final processed dataset rather than manually recreated in the frontend.

8. User Journey

- User opens the website.
- User navigates to the disease detection page.
- User selects a plant/leaf image.
- The image is displayed for preview.
- User clicks **Detect Disease**.
- Frontend sends the image to the backend.
- Backend communicates with the Python ML service.
- ML service preprocesses the image and runs the trained model.
- Prediction and confidence are returned.
- Frontend displays the result, disease information, and general recommendations.

9. Functional Requirements

ID	Requirement
FR-01	System shall allow users to upload JPG, JPEG, and PNG plant images.
FR-02	System shall preview the selected image before prediction.
FR-03	System shall validate file type and configured upload-size limits.
FR-04	System shall expose a prediction API for image inference.
FR-05	ML service shall preprocess the image before inference.
FR-06	ML service shall classify the image into one of the supported 38 classes.
FR-07	System shall return the predicted class and confidence score.
FR-08	System shall identify healthy classes where present in the dataset.
FR-09	Frontend shall display crop, disease, confidence, and status.
FR-10	System shall display disease information and general preventive guidance.
FR-11	System shall show loading state during inference.
FR-12	System shall handle validation, network, backend, and ML errors.

10. System Architecture

The proposed architecture separates the user interface, application backend, and machine-learning inference service.

Layer	Technology	Responsibility
Frontend	React + Vite	Upload image, UI state, display prediction.
Application Backend	Node.js + Express	Request handling, validation, API gateway.
ML Service	Python + FastAPI/Flask	Preprocessing, model loading, inference.
Model	TensorFlow/PyTorch	38-class image classification.
Storage	None for MVP	No persistent application database required.

Logical flow: User → React Frontend → Node/Express Backend → Python ML API → Preprocessing → Trained Model → Prediction + Confidence → Frontend Result.

11. Frontend Requirements

The frontend will use React.js with Vite and CSS. The interface should be simple enough for a non-technical user.

- **Home:** Project introduction, call-to-action, supported crops/classes, and how it works.
- **Detection:** Upload control, image preview, validation, and prediction button.
- **Result:** Uploaded image, crop, disease, confidence, status, description, and recommendations.
- **About:** Project objective, technologies, ML overview, and dataset information.

12. Backend Requirements

The Node.js/Express backend will act as the application API layer and communicate with the Python ML service.

- Receive multipart image uploads.
- Validate incoming requests.
- Forward images to the ML service.
- Return normalized prediction responses to the frontend.
- Handle errors and service timeouts.
- Apply file-upload restrictions.

13. Python ML Service Requirements

- Load the trained model when the service starts.
- Receive an image through an inference endpoint.
- Validate and preprocess the image.
- Generate class probabilities.
- Select the highest-probability class.
- Return class name, confidence, crop, and status.
- Return structured errors if inference fails.

14. API Specification

Endpoint	Method	Input	Purpose
/api/predict	POST	multipart/form-data image	Predict crop disease through the backend.
/predict	POST	Image	Python ML inference endpoint (internal service).

14.1 Example Prediction Response

```
{"success": true, "prediction": {"crop": "Tomato", "disease": "Late Blight",
"confidence": 94.7, "status": "Diseased"}}
```

15. Machine Learning Requirements

- Use the prepared 38-class labeled crop disease dataset.
- Perform dataset cleaning and validation.
- Create training, validation, and test splits.
- Apply appropriate image resizing and normalization.
- Use data augmentation when appropriate.
- Train a CNN or transfer-learning model.
- Use checkpointing and early stopping when appropriate.
- Save the final trained model for inference.
- Keep class-to-index mapping synchronized with the model.

16. ML Training Pipeline

The recommended training pipeline is:

Dataset → Cleaning → Train/Validation/Test Split → Preprocessing → Augmentation → Model Training → Validation → Hyperparameter Tuning → Final Evaluation → Save Model → Deploy Model

17. Model Evaluation

Metric	Purpose
Accuracy	Measures overall classification correctness.
Precision	Measures correctness of positive predictions.
Recall	Measures the ability to identify relevant samples.
F1 Score	Balances precision and recall.
Confusion Matrix	Shows class-level prediction errors.
Per-Class Metrics	Important for identifying weak disease classes in a 38-class problem.

The final model should be evaluated on a held-out test set that is not used for model fitting. Actual metric values should be inserted into the final technical/project report after training.

18. Database Requirements

No database is required for the MVP. The prediction flow can operate using the frontend, Express backend, Python ML service, and saved model.

A database may be introduced later for user accounts, prediction history, reports, disease records, and analytics.

19. Non-Functional Requirements

Category	Requirement
Performance	Prediction response should be reasonable for the deployed model and hardware.
Usability	A user should be able to complete detection without technical knowledge.
Responsiveness	Interface should work on desktop, tablet, and mobile.
Reliability	System should show meaningful errors when services fail.
Maintainability	Frontend, backend, and ML components should remain separated.
Security	Uploaded files and API inputs must be validated and restricted.

20. Security Requirements

- Restrict allowed file types and MIME types.
- Limit maximum upload size.
- Do not execute uploaded files.
- Validate API input on the server side.
- Use HTTPS in production.
- Store secrets in environment variables.
- Do not commit .env files or credentials to GitHub.

21. Suggested Project Structure

```
crop-disease-detection/  
  ■■■ frontend/  
  ■ ■■■ src/  
  ■ ■■■ public/  
  ■ ■■■ package.json  
  ■ ■■■ vite.config.js  
  ■■■ backend/  
  ■ ■■■ routes/  
  ■ ■■■ controllers/  
  ■ ■■■ middleware/  
  ■ ■■■ server.js  
  ■ ■■■ package.json  
  ■■■ ml-service/  
  ■ ■■■ model/  
  ■ ■■■ dataset/  
  ■ ■■■ utils/  
  ■ ■■■ app.py  
  ■ ■■■ class_names.json  
  ■ ■■■ requirements.txt  
  ■■■ README.md  
  ■■■ .gitignore
```

22. Training vs Production

Training and production inference are separate processes. The large dataset is used during training in Jupyter or Google Colab. Once the model is trained and evaluated, the saved model is deployed to the Python ML service.

Training: Dataset → Jupyter/Colab → Train → Evaluate → Save Model

Production: User Image → ML API → Saved Model → Prediction

The production application does not retrain the model for every uploaded image.

23. MVP Definition

- React/Vite web interface.
- Image upload and preview.
- Express backend.
- Python ML inference service.
- Trained 38-class model.
- Disease prediction.
- Confidence score.
- Disease information.
- General recommendations.
- Error handling.
- Responsive design.

24. Future Enhancements

- User authentication and prediction history.

- Downloadable PDF reports.
- Top-3 predictions.
- Multiple-image analysis.
- Improved model architectures.
- Native mobile application.
- Multilingual interface.
- Voice-based interaction.
- Weather and agricultural data integration.
- Disease severity estimation.
- Leaf-region segmentation.
- Field-level monitoring using drone or satellite imagery.

25. Risks and Limitations

Risk / Limitation	Mitigation
Dataset bias	Use diverse data and evaluate on representative unseen images where possible.
Similar disease symptoms	Improve dataset quality and perform per-class evaluation.
Poor image quality	Give users guidance to upload clear images with adequate lighting.
High confidence but incorrect prediction	Communicate that model confidence is not a guarantee of diagnosis.
Real-world domain shift	Test on images captured outside the training environment.

26. Acceptance Criteria

- User can open the website successfully.
- User can upload a supported image format.
- Selected image appears in preview.
- User can start detection.
- Frontend communicates successfully with the backend.
- Backend communicates successfully with the ML service.
- ML service loads the trained model.
- Model produces a prediction from a valid image.
- Frontend displays crop, disease, confidence, and status.
- Invalid uploads produce an appropriate error.
- Application works on desktop and mobile layouts.
- Final model is evaluated on a held-out test set and results are documented.

27. Project Milestones

Phase	Deliverable
1	Dataset collection and verification
2	Dataset cleaning and preprocessing
3	Train/validation/test preparation
4	Model development
5	Model training and tuning
6	Model evaluation
7	Python ML API
8	Node/Express backend
9	React frontend
10	Frontend-backend-ML integration
11	Testing and bug fixing
12	Deployment
13	Final project documentation

28. Technology Stack

Category	Technology
Frontend	React.js + Vite
Styling	CSS
HTTP Client	Axios
Backend	Node.js + Express.js
ML Service	Python + FastAPI/Flask
ML Framework	TensorFlow or PyTorch
Model	CNN / Transfer Learning
Dataset	Crop Disease Dataset — 38 Classes
Training	Jupyter Notebook / Google Colab
Version Control	Git + GitHub
Database	None for MVP

29. Final Product Definition

The final MVP is a web-based AI system that allows a user to upload a crop leaf image and receive an automated prediction from a trained 38-class crop disease classification model.

The application separates the user interface, application backend, and ML inference service. This separation makes it possible to improve the model independently from the frontend and backend.

Final architecture: User → React/Vite → Node/Express → Python ML API → Image Preprocessing → Trained 38-Class Model → Disease + Confidence → Web Result.

Appendix A — Example User-Facing Result

Field	Example
Crop	Tomato
Disease	Late Blight
Confidence	94.7%
Status	Diseased
Information	Short description of the detected condition.
Recommendation	General preventive/management guidance.

Appendix B — Important Project Note

Model accuracy, precision, recall, F1 score, class distribution, inference latency, and the final 38-class list should be updated with measured project results before final academic submission. The PRD defines requirements; it should not claim performance numbers that have not yet been experimentally verified.